

Adaptive Learning Path Builder

Candidate-facing project brief, API schema contract, and submission requirements

Target profile

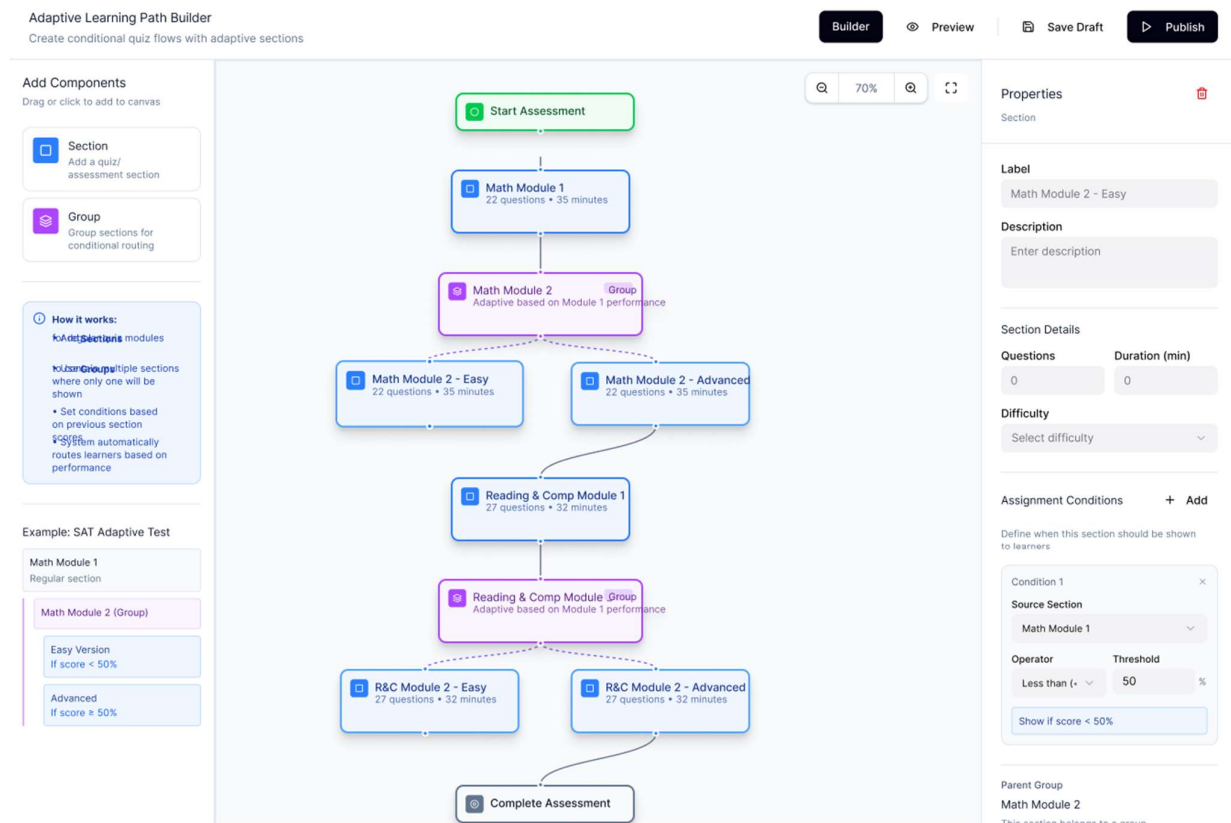
Mid-level full-stack developer (2 to 3 years experience)

Expected effort

One working day

Primary focus

Interactive UI, API design, graph persistence, and conditional learning logic



Visual reference for the builder screen. Candidates should keep the overall layout, hierarchy, and feel close to this design.

1. Objective

Build a web application that allows an admin or curriculum designer to create an adaptive learning path by arranging content nodes on a canvas and defining progression logic between them. **The provided design should be treated as the UI reference.**

2. What we want to evaluate

- Ability to implement a polished frontend in TypeScript using React or Angular.
- Ability to design and implement clean backend APIs in Python or Java.
- Ability to model graph-like data: nodes, edges, positions, and conditional rules.

- Ability to build drag-and-drop and connector interactions with clear state management.
- Ability to document setup, seed data, and testing so the project is easy to verify.

3. Functional scope

A. Left panel: available content

- Load available content from an API and display it in the left panel.
- Each item must show title, short description, content type, and approximate duration.
- Supported types: unit and assessment.

B. Canvas builder

- Allow users to drag content items from the left panel onto the central canvas.
- Allow nodes to be repositioned after placement.
- Support multiple nodes on the canvas.

C. Flow connections

- Allow users to create directed connections between nodes by drawing lines between them.
- Support both linear and branching flows.
- Persist all saved node positions and connections.

D. Conditional logic on connections

- When a connection is selected, allow the user to define progression rules.
- Assessment conditions can be based on completion, passing status, or score / score range.
- Unit conditions can be based on completion, minimum time spent, or percentage completion.

E. Properties panel

- When a node is selected, show editable node properties on the right.
- When an edge is selected, show editable condition properties on the right.

F. Save and reload

- Provide an API to save the learning path.
- Reloading the path should restore nodes, positions, labels, and rules.

4. Technology expectations

Layer	Expectation
Frontend	TypeScript. React preferred, Angular acceptable.
Backend	Python or Java preferred.
Persistence	SQLite, H2, JSON-file persistence, or another lightweight approach.
Libraries	Reasonable use of graph / drag-and-drop libraries is allowed. Document any library choices in the README.

5. API contract the candidate should follow

The candidate should use the provided JSON Schema files as the contract for structuring the APIs. At minimum, the project should implement the endpoints below.

Method	Endpoint	Purpose	Schema / Notes
GET	/api/components	Return all draggable content for the left-side panel.	Response should match available-content.schema.json
POST	/api/learning-paths	Save a learning path built in the canvas.	Request body should match learning-path.schema.json
GET	/api/learning-paths/{id}	Load a previously saved learning path.	Response shape should match learning-path.schema.json
Optional POST	/api/learning-paths/{id}/evaluate	Evaluate next node using learner context.	Optional extension; useful for stronger submissions

6. Schema specification for GET /api/components

Response wrapper: an object with items[] and totalCount.

Field	Type	Required	Notes
items	array	Yes	Array of content components displayed in the left panel.
totalCount	integer	Yes	Total number of available content items.

Component object fields:

Field	Type	Required	Notes
id	string	Yes	Unique component identifier.
title	string	Yes	Display title in the left panel and when dropped on the canvas.
shortDescription	string	Yes	One-line or two-line supporting text.
type	enum	Yes	Allowed values: unit, assessment.
approximateDurationMinutes	integer	Yes	Approximate duration shown in UI.
metadata.assessment.maxScore	integer	Required for assessment	Maximum score for assessment nodes.
metadata.assessment.passingScore	integer	Required for assessment	Passing score threshold used by passed conditions.
metadata.unit.recommendedMinutes	integer	Optional for unit	Suggested duration or effort target for the unit.

Example response

```
{
  "items": [
    {
      "id": "cmp-assess-math-1",
      "title": "Math Module 1 Assessment",
      "shortDescription": "Baseline math diagnostic used to route learners.",
      "type": "assessment",
      "approximateDurationMinutes": 35,
      "metadata": {
        "assessment": {
          "maxScore": 100,
          "passingScore": 50
        }
      }
    }
  ],
  "totalCount": 1
}
```

7. Conditional rule model

Source type	Allowed metrics	Example operator	Example
Assessment	completion, passed, score, score_range	eq, gte, between	score between 0 and 49 routes learner to Easy module
Unit	completion, time_spent_minutes, percentage_completion	eq, gte	percentage_completion >= 80 routes learner forward

8. Deliverables

- Public source repository with frontend, backend, README, seed data, and tests.
- Implementation of the API contract using the provided schema files.
- A working builder UI that loads content from the API, supports node placement, supports connections, and allows editing rules.
- Instructions to run the project locally.
- Test results or screenshots showing tests were executed.

9. Submission checklist

1. Repository link
2. Time spent
3. Assumptions or tradeoffs
4. Setup instructions
5. Test execution evidence

10. Evaluation criteria

Category	Weight
UI quality and fidelity to design	30%
Frontend interaction quality	25%

Category	Weight
Backend / API design	20%
Conditional logic and data modeling	15%
Testing and documentation	10%

11. Included files

- available-content.schema.json — JSON Schema for GET /api/components response.
- available-content.example.json — example response for GET /api/components.
- learning-path.schema.json — JSON Schema for saving and loading learning paths.
- learning-path.example.json — example saved learning path payload.
- UI mockup to be followed and design spec files